

Group 1

Members:

Zane Garvey, Michael Zandonella, Lapatrada Liawpairoj, Sawyer Fedderly

What Our App Will Do:

Our project will help users identify movies/TV shows from partial or remembered quotes. Users can enter a line of dialogue, and the app will search through a movie script dataset, rank potential matches using a text similarity algorithm, and display the most likely results. Each result is clickable. When a user selects a movie, the app fetches detailed information such as ratings, a plot summary, and the movie poster from the TMDb API. This application will include dynamic routes, interactive stateful components, and a responsive, polished user interface.

API Link:

<https://developer.themoviedb.org/docs/getting-started>

API Description:

We will be using the TMDb (The Movie Database) API, a RESTful web service. Our app will primarily call GET endpoints such as `/3/search/movie` (title-based search) and `/3/movie/{movie_id}` (detailed movie lookup) to retrieve movie metadata in JSON format.

When our quote-matching system produces ranked candidate movies, we will resolve each candidate to a TMDb movie_id (either by searching title via `/search/movie` or, if we have an IMDb ID, by using TMDb's "Find By ID" endpoint `/3/find/{external_id}`).

The API returns data such as title, release date/year, overview (plot summary/description), genres, and vote_average (TMDb's rating-like score), along with poster_path values. To display posters, we will construct full image URLs using TMDb's documented image URL process (base URL + size + file path).

We will use this metadata to populate a `/movie/:tmdbId` detail page when a user selects a ranked quote match. Our application will send the selected movie's TMDb ID (and our API key / auth token) as part of the HTTP GET request, and it will use the structured JSON response to render the movie's information into the UI. No persistent data will be sent back to TMDb aside from the read-only lookup requests.

UI Organization Description:

Routes / Pages:

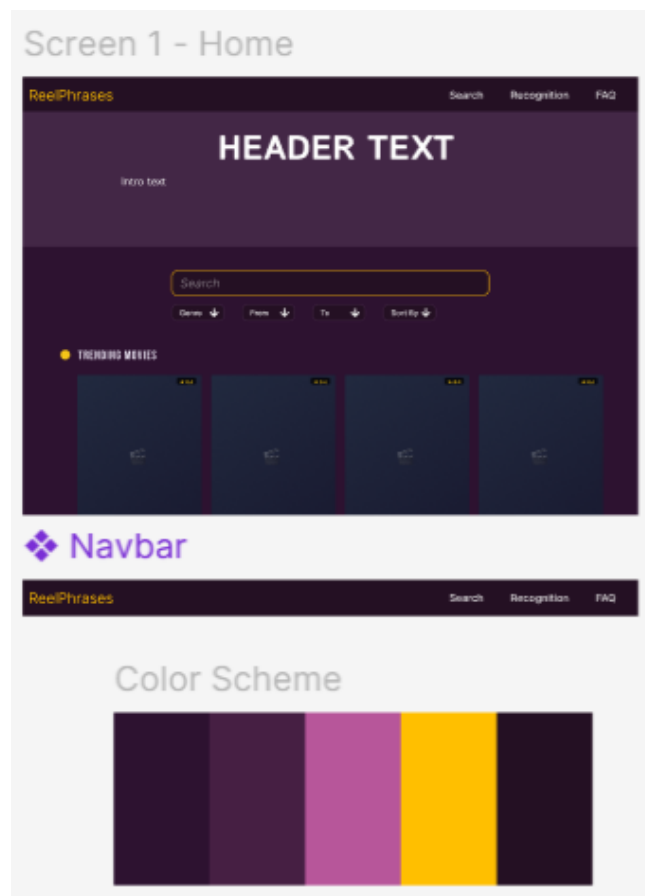
- Home (/): Hero section with app name, tagline, and a centered quote search bar. Includes a navbar with links to Search and FAQ. (option: a small text or FAQ to guide/help new users)
- Search (/search): Full quote input with optional filters — Genre, Year Range (years), and Sort By. Clean filter row beneath the search bar for quick refinement.
- Results (/results): Ranked list of matching movies displayed as cards. Each card shows a movie poster, title, release year, and a short matched quote snippet. Cards are clickable and navigate to the Movie Details page.
- Movie Details (/movie/:tmdbId): Full movie info pulled from TMDb (poster, title, plot summary, genres, rating, and release year).

Main interactions:

Submit quote → fetch matches → display ranked results

Click a result → navigate to the details page and fetch full metadata

Mocks of Major Activities:



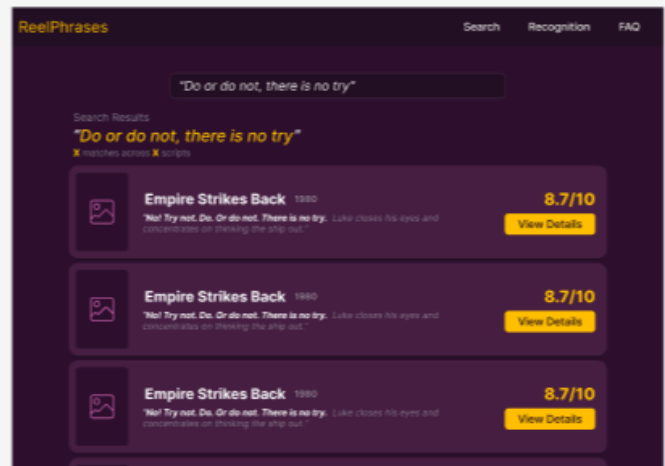
Screen 2 - Search



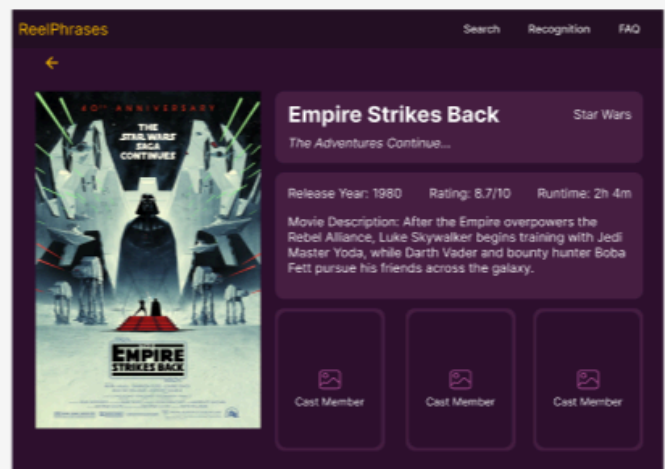
Screen 2 - Search (Loading)



Screen 3 - Results



Screen 4 - Movie Details



Division of Labor:

Zane - Quote search + backend API

- Build a preprocessing pipeline for the dataset
- Implement /api/search endpoint
- Implement relevance scoring + returning ranked candidates

Michael - Results page + state handling

- Implement /results route, display ranked search results, and connect each result to each movie detail page
- Integrate frontend with /api/search endpoint
- Manage search query states

Lapatrada - Front-end routes + UI/UX

- Build and design pages: Home (/), Search (/search), and Results (/results) in Figma– plan to focus on desktop, then mobile responsiveness.

- Establish the component library and shared UI patterns (buttons, search bar, filter dropdowns, movie cards) for consistency across pages.
- Design system: color palette and typography

Sawyer: Movie details integration + favorites

- TMDB integration (search/detail fetch)
- Movie detail page '/movie': API route + DB schema + UI page